

Python: Intermediate-Level Python Programming

Assignment study PDF — English

1. Intermediate Python topics

Intermediate programming moves beyond basic syntax into comprehensions, functions, exceptions, modules, decorators, generators, context managers, object-oriented features, typing and concurrency.

2. List comprehensions

Example: `squares = [x*x for x in range(10)]`. Comprehensions provide a concise way to create collections from iterable data.

3. Functions and arguments

Python supports positional arguments, keyword arguments, default values, `*args` and `**kwargs`. Understanding argument passing is important for reusable code.

4. Exceptions

Use `try/except` to handle expected runtime errors. Use `finally` for cleanup that must happen whether an exception occurs or not.

5. Decorators

A decorator wraps or modifies a function or class without changing the original implementation directly. Decorators are commonly used for logging, authorization, caching and timing.

6. Generators

Generators use `yield` to produce values lazily. They are useful when processing large sequences because they avoid storing all results in memory at once.

7. Context managers

The `with` statement is used for resource management, such as opening files. Context managers help guarantee cleanup even when errors occur.

8. OOP and magic methods

Classes can define special methods such as `__init__`, `__str__` and `__len__`. These methods integrate custom objects with normal Python operations.

9. Practice

Build small programs that combine functions, exception handling, file I/O, classes, decorators or generators. Focus on readable, modular and testable code.